

```

        turtle.setheading(initialHeading + angleCh)
        turtle.forward(size * xCh)
        turtle.left(90)
        turtle.forward(size * yCh)
        turtle.right(90)

        # Make the recursive call:
        drawFractal(shapeDrawFunction, size * sizeCh, specs, maxDepth,
                    depth + 1)

if DRAW_FRACTAL == 1:
    # Four Corners:
    drawFractal(drawFilledSquare, 350,
                [{'sizeChange': 0.5, 'xChange': -0.5, 'yChange': 0.5},
                 {'sizeChange': 0.5, 'xChange': 0.5, 'yChange': 0.5},
                 {'sizeChange': 0.5, 'xChange': -0.5, 'yChange': -0.5},
                 {'sizeChange': 0.5, 'xChange': 0.5, 'yChange': -0.5}], 5)
elif DRAW_FRACTAL == 2:
    # Spiral Squares:
    drawFractal(drawFilledSquare, 600, [{'sizeChange': 0.95,
                                          'angleChange': 7}], 50)
elif DRAW_FRACTAL == 3:
    # Double Spiral Squares:
    drawFractal(drawFilledSquare, 600,
                [{'sizeChange': 0.8, 'yChange': 0.1, 'angleChange': -10},
                 {'sizeChange': 0.8, 'yChange': -0.1, 'angleChange': 10}])
elif DRAW_FRACTAL == 4:
    # Triangle Spiral:
    drawFractal(drawTriangleOutline, 20,
                [{'sizeChange': 1.05, 'angleChange': 7}], 80)
elif DRAW_FRACTAL == 5:
    # Conway's Game of Life Glider:
    third = 1 / 3
    drawFractal(drawFilledSquare, 600,
                [{'sizeChange': third, 'yChange': third},
                 {'sizeChange': third, 'xChange': third},
                 {'sizeChange': third, 'xChange': third, 'yChange': -third},
                 {'sizeChange': third, 'yChange': -third},
                 {'sizeChange': third, 'xChange': -third, 'yChange': -third}])
elif DRAW_FRACTAL == 6:
    # Sierpiński Triangle:
    toMid = math.sqrt(3) / 6
    drawFractal(drawTriangleOutline, 600,
                [{'sizeChange': 0.5, 'yChange': toMid, 'angleChange': 0},

```